

Arquitetura Hexagonal: O que é, pra que serve e como pode salvar seu código do caos

O uso de portas e adaptadores na Arquitetura Hexagonal ajuda a organizar as interações, mantendo a lógica de negócios desacoplada e independente das tecnologias externas. - Matthias Noback



RAFAEL MIGUEL
NOV 01, 2024



21



5



1

Compartilhar

Neste artigo, vamos explorar a **arquitetura hexagonal** e como ela pode transformar a forma de estruturar sistemas. Veremos como a separação entre o núcleo da aplicação e o mundo externo traz benefícios como **flexibilidade**, **robustez**, e **facilidade de manutenção**. Vamos falar sobre **portas**, que conectam a lógica de negócios às interações externas, e **adaptadores**, que fazem a ponte entre o núcleo e as tecnologias que o cercam. Ao final, você entenderá como essa abordagem pode preparar seu sistema para evoluir sem dores de cabeça, permitindo trocas de tecnologias externas sem mexer na lógica central.

O Crescimento dos Sistemas ao Longo do Tempo

A maioria dos projetos começa pequena, com uma arquitetura simples e clara. Muitas vezes, o sistema é dividido em camadas bem conhecidas:

- **Camada de Apresentação (UI):** Responsável pela interface do usuário, seja ela web, desktop ou mobile, onde o usuário interage diretamente com o sistema.
- **Camada de Aplicação/Serviços:** Aqui ficam as regras de negócio, onde a lógica do sistema é coordenada, processando as operações e conectando a interface com os dados.
- **Camada de Acesso a Dados (DAO):** É a parte que lida com a comunicação com o banco de dados, enviando e recebendo as informações necessárias para que a lógica de negócios funcione.
- **Camada de Banco de Dados:** Onde os dados são efetivamente armazenados e recuperados.

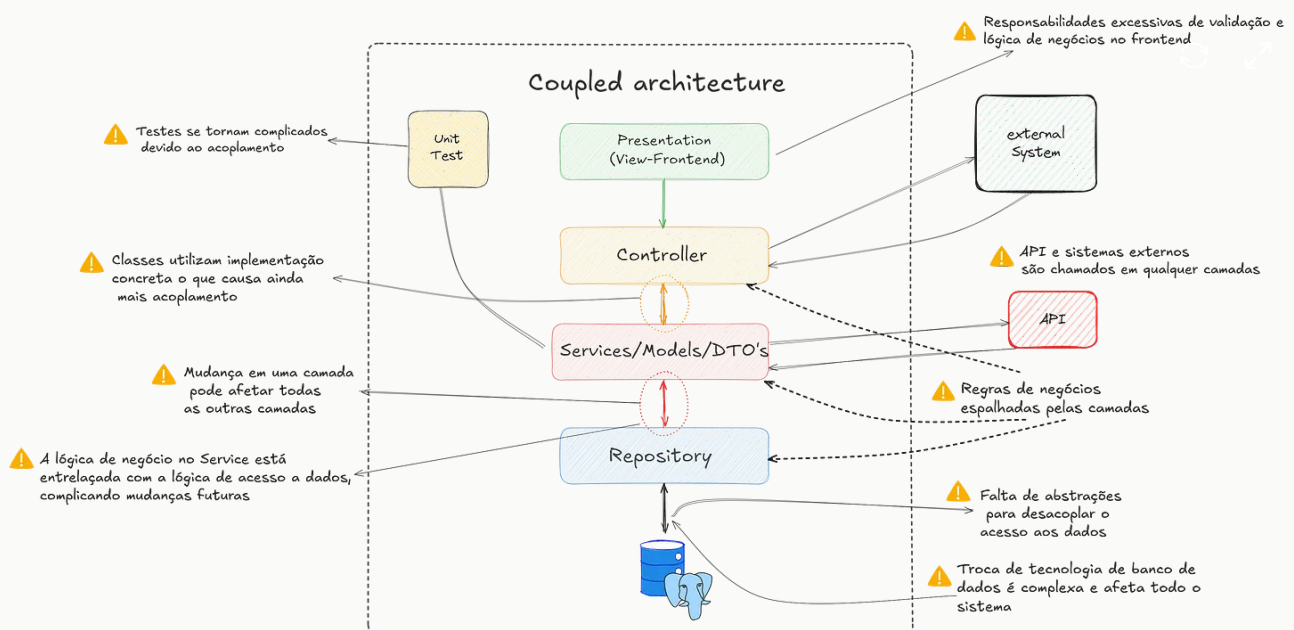
Essa divisão de responsabilidades é bastante comum e serve bem a muitos projetos, principalmente no começo, quando tudo é simples e o número de funcionalidades é limitado. No entanto, à medida que o sistema cresce e se torna mais complexo, essa organização pode se transformar em um grande desafio.

Imagine que o seu sistema já esteja em operação há alguns anos. Ele começou pequeno, mas agora tem múltiplos módulos, integrações com outros sistemas, e uma base de usuários crescente. Cada vez que você precisa adicionar uma nova funcionalidade, percebe que ela vai impactar várias dessas camadas ao mesmo tempo.

Por exemplo, se for necessário alterar uma regra de negócios, essa mudança pode impactar diretamente a **camada de aplicação** e, conseqüentemente, a **UI** (que precisa exibir os dados corretamente), além de também forçar alterações na **camada de acesso a dados**. Essa interdependência cria um ambiente onde cada mudança precisa ser cuidadosamente ajustada em várias partes do sistema, tornando o processo lento e propenso a erros. É aqui que o sistema começa a perder sua agilidade.

Com o tempo, essa estrutura rígida se torna uma armadilha. A necessidade de manter todas as camadas funcionando perfeitamente em conjunto acaba tornando o sistema mais **frágil e difícil de modificar**. A equipe passa mais tempo tentando entender como as mudanças vão impactar as diferentes partes do código do que efetivamente inovando e agregando valor. Isso também afeta os testes – testar uma parte do sistema pode significar ter que executar várias outras partes, o que aumenta a complexidade e o tempo gasto.

Além disso, quando você depende dessa divisão clássica em camadas, pode acabar se tornando refém das tecnologias escolhidas no início do projeto. Trocar o banco de dados, por exemplo, pode se tornar um pesadelo, já que ele está tão integrado com o restante do sistema que mexer nele pode exigir refatorações em várias outras partes.



O que parecia uma organização simples no início agora se transforma num verdadeiro fardo, dificultando a evolução do sistema. A separação entre as camadas, que deveria ser um benefício, acaba gerando **alto acoplamento** — ou seja, as camadas dependem tanto umas das outras que uma mudança pequena em uma delas pode impactar profundamente as outras.

O Impacto do Acoplamento no Trabalho das Equipes de Engenharia

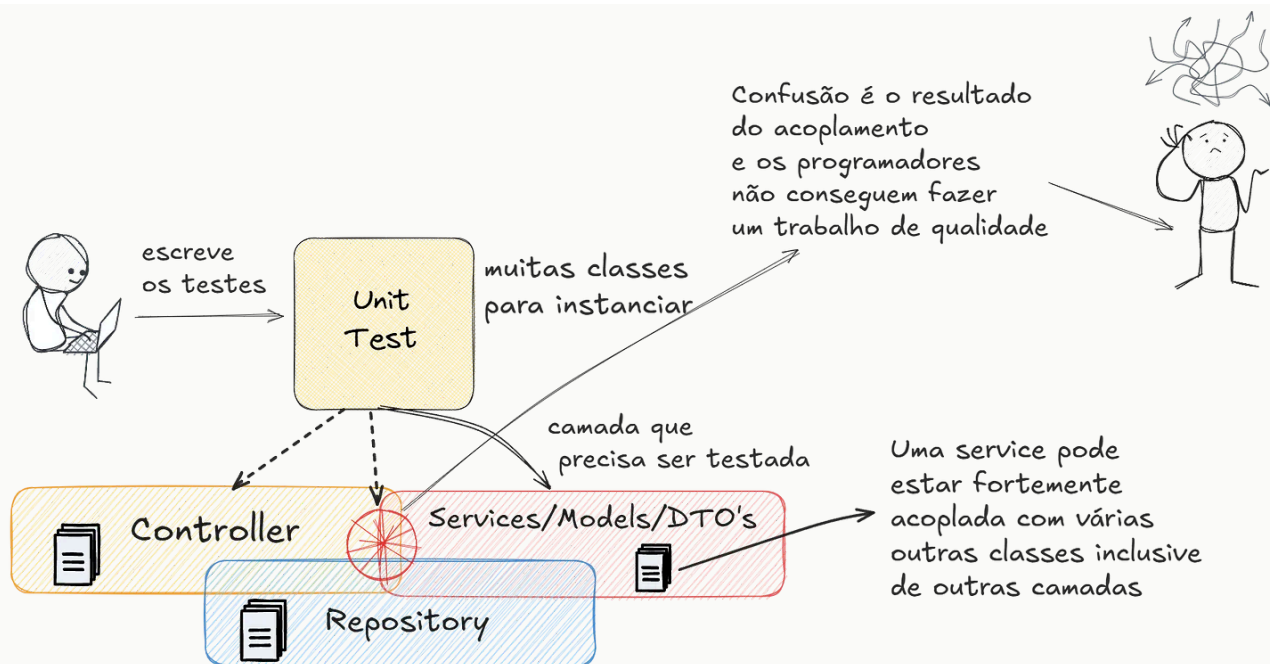
Já passou por um projeto onde qualquer mudança parecia um campo minado?

Aquele sistema onde, ao longo do tempo, as partes ficaram tão entrelaçadas que uma simples alteração podia causar um efeito dominó, quebrando outras áreas que nem estavam relacionadas? Se sim, você sabe como isso pode tornar o trabalho da equipe um verdadeiro pesadelo.

Testes se Tornam um Pesadelo

Quando o sistema é **fortemente acoplado**, testar uma nova funcionalidade vira uma tarefa muito mais complicada do que deveria ser. Pense na seguinte situação: você precisa alterar uma regra de negócio simples. O problema é que, como todas as camadas do sistema estão interligadas, **não dá para testar essa mudança isoladamente**. Para garantir que tudo funcione, você precisa checar as interações com a interface de usuário, verificar se o acesso aos dados ainda está correto e, claro, garantir que o banco de dados está respondendo como esperado.

Já passou por isso? Esse tipo de dependência cria uma cadeia que torna cada mudança algo **arriscado**. De repente, aquela pequena alteração está quebrando partes do sistema que você nem imaginava que seriam afetadas. E o pior: os testes ficam **mais demorados e complexos**, forçando a equipe a gastar um tempo enorme criando testes integrados para garantir que tudo ainda funciona de ponta a ponta. **Já sentiu que está passando mais tempo corrigindo bugs do que criando novas funcionalidades?** Isso é um sinal clássico de acoplamento exagerado.



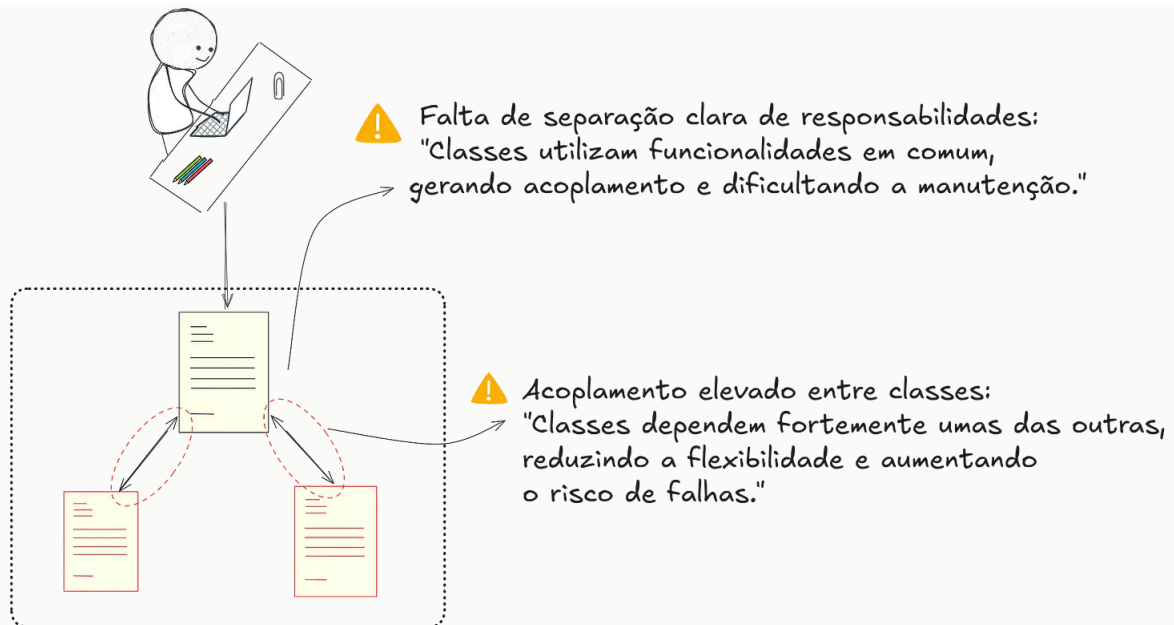
Legenda:

-  representa o acoplamento entre classes (camadas)
-  seta pontilhada representa que precisamos instanciar/injetar nos testes de unidade as classes por causa do acoplamento entre as elas
-  representa as classes dentro de cada camada

Aquela linha tênue entre testes de unidade e testes de integração começa a desaparecer. O que deveria ser um teste unitário simples, focado em um comportamento específico, acaba se tornando um teste de integração disfarçado. Isso porque o acoplamento força a testar **múltiplas camadas juntas** – como a lógica de negócios, repositórios de dados, e até a interface. O resultado? **Testes mais lentos**, difíceis de manter, e um risco crescente de surgirem novos bugs.

Lançar Novas Funcionalidades Fica Lento e Arriscado

E as novas funcionalidades? Quando o sistema está acoplado, **lançar algo novo vira um teste de paciência**. Já passou por isso? Uma simples mudança na lógica de negócios pode fazer você mexer na interface de usuário, nos serviços e nos dados. Algo que deveria ser resolvido em poucos dias acaba se arrastando por semanas, ou até meses.



Você já ficou inseguro sobre o impacto das alterações? Isso acontece porque, em um sistema acoplado, você nunca tem certeza do que vai quebrar. Às vezes, ao adicionar uma nova funcionalidade, algo que estava funcionando perfeitamente acaba parando – e o pior: você pode nem perceber isso de imediato. O problema pode aparecer em uma camada completamente diferente. O que deveria ser uma vitória para o time acaba gerando **ansiedade e frustração**. O que vai quebrar dessa vez?

Corrigir Bugs Demora Muito Mais

E quando surgem bugs? Acha difícil descobrir de onde eles vêm? Isso acontece muito em sistemas acoplados. O comportamento de uma parte do sistema acaba dependendo de várias outras. **Para encontrar a causa raiz de um bug, os engenheiros têm que investigar diversas camadas, o que consome um tempo precioso.**

E, mesmo quando o problema é encontrado, **a solução raramente está em um único lugar**. Para corrigir um bug, você acaba ajustando várias partes do código, o que aumenta o risco de criar **novos problemas** enquanto resolve o original. Já passou por isso? Parece um ciclo infinito: você corrige um bug, e outros aparecem. A equipe vai se frustrando, e a entrega da solução acaba demorando muito mais do que deveria.

A Qualidade do Código Deteriora

Com o tempo, sistemas acoplados começam a perder a qualidade do código. **Já se pegou mexendo em áreas do sistema que você nem conhece muito bem, só para fazer uma mudança simples?** Isso acontece porque, com tantas dependências,

qualquer alteração exige mexer em várias partes, levando a "gambiarrras" e soluções rápidas.

A pressão por entregar novas funcionalidades rapidamente impede que o time tenha tempo para **refatorar** ou melhorar o código existente. Com o tempo, o código vai se tornando cada vez mais **confuso** e **difícil de entender**. E quanto mais isso acontece, mais difícil fica mantê-lo.

Arquitetura Hexagonal: A Solução para o Acoplamento Excessivo

Depois de entendermos os desafios que surgem com classes e camadas fortemente acopladas, é importante olharmos para uma abordagem que foi criada justamente para resolver esses problemas. A **Arquitetura Hexagonal** foi proposta como uma forma de criar sistemas mais flexíveis, fáceis de manter e resistentes a mudanças, eliminando os ciclos de frustração causados pelo acoplamento excessivo entre camadas e classes.

Essa arquitetura foi introduzida por **Alistair Cockburn** no início dos anos 2000 e, em sua essência, foca em **desacoplar** a lógica de negócios (ou seja, as regras e funcionalidades principais do sistema) de qualquer tecnologia ou detalhes de implementação, como frameworks, bancos de dados ou interfaces de usuário.

A Arquitetura Hexagonal é sobre isolar o núcleo da aplicação, mantendo-o independente das interfaces de comunicação externas, de modo que ele possa evoluir e ser testado sem dependências.

A ideia é que o sistema seja construído em torno de um núcleo sólido de regras de negócios, que não dependa diretamente de componentes externos.

Resolvendo problemas!

Imagine que, em vez de ter classes e camadas fortemente interligadas, onde uma simples mudança afeta múltiplas partes do sistema, o seu código fosse organizado de forma que a lógica de negócios estivesse **isolada** de tudo o que é externo. Na Arquitetura Hexagonal, essa lógica central (o coração da aplicação) não precisa "saber" como os dados são armazenados ou de onde vêm as requisições.

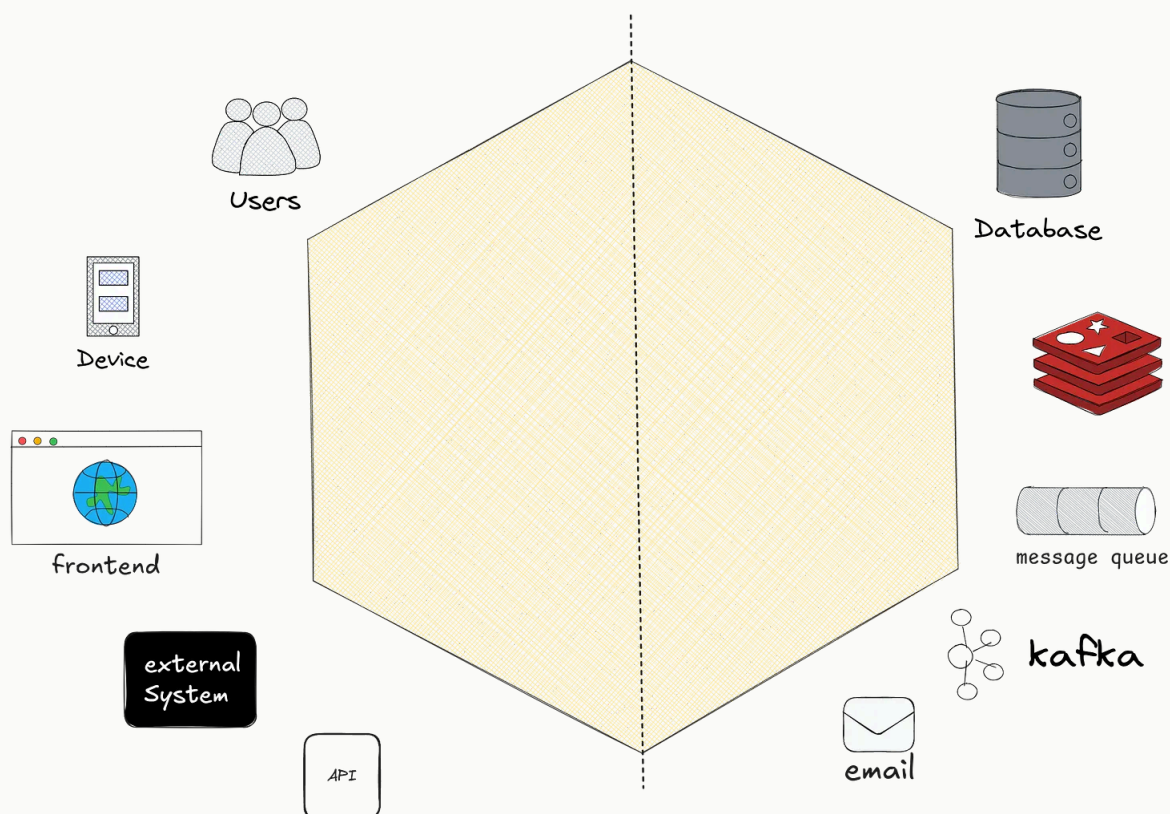
Essa separação nos protege dos principais problemas que enfrentamos em sistemas com camadas fortemente acopladas, como:

- **Facilidade de Testes:** Como a lógica de negócios está desacoplada de tecnologias externas, podemos testar suas funcionalidades de maneira

isolada, sem precisar instanciar múltiplas dependências ou camadas. Isso elimina a necessidade de mocks complicados e garante que o comportamento da aplicação seja validado de forma rápida e eficaz.

- **Adição de Novas Funcionalidades sem Impacto em Outras Partes do Sistema:** Na Arquitetura Hexagonal, adicionar uma nova funcionalidade ou mudar uma regra de negócios não requer mudanças nas camadas de armazenamento de dados ou na interface do usuário, pois essas partes estão isoladas. Isso reduz o risco de bugs em outras áreas ao introduzir algo novo.
- **Facilidade de Manutenção e Refatoração:** Como cada parte do sistema tem responsabilidades bem definidas e é menos dependente de outras camadas, fica muito mais fácil fazer refatorações ou ajustes sem causar um efeito cascata. Quando um bug é encontrado, ele pode ser corrigido no contexto isolado da funcionalidade, sem afetar outras áreas.
- **Redução no Tempo de Entrega:** A separação clara de responsabilidades e o isolamento das classes de negócio diminuem o tempo necessário para ajustar funcionalidades e corrigir bugs, impactando positivamente o tempo de entrega.

Ao remover as dependências diretas entre as camadas, a Arquitetura Hexagonal nos permite construir um sistema onde cada parte possa evoluir de maneira independente. Isso significa que não importa se estamos trocando o banco de dados ou mudando a interface com o usuário, a lógica de negócios continua intacta, sem precisar de grandes alterações.



A imagem acima mostra a casca apenas e você pode ver todo o mundo externo está em volta do hexágono, em breve vamos detalhar mais e completar essa imagem e as interações!

Nos próximos tópicos, vamos mergulhar nos conceitos principais que compõem a Arquitetura Hexagonal e entender como ela organiza esses componentes para que possamos livrar o sistema dos problemas de acoplamento.

O Núcleo da Aplicação

A **Arquitetura Hexagonal** é construída em torno da ideia de proteger o núcleo da aplicação, onde estão as **regras de negócio**. Esse núcleo é a parte mais **valiosa** do sistema, pois define **o que o sistema faz** — suas funcionalidades, suas decisões, e como os dados são refinados.

O núcleo da aplicação é uma camada de código que contém as **decisões de negócios** e **as regras mais críticas**. É importante entender que essa parte do sistema deve ser **totalmente isolada** das preocupações externas, como frameworks de interface do usuário, bancos de dados ou qualquer outra infraestrutura. A lógica de negócios deve ser independente de como os dados são armazenados ou exibidos.

Por exemplo, imagine um sistema de vendas. As regras que dizem como calcular o preço final de uma venda com base em descontos, taxas, ou promoções devem estar no núcleo. Essas regras não deveriam depender de como os dados da venda são armazenados no banco de dados ou de como são exibidos para o usuário final.

A grande sacada da Arquitetura Hexagonal é que ela **protege o núcleo** das influências externas. Isso significa que, se um dia você precisar trocar o banco de dados ou mudar a forma como os usuários interagem com o sistema, o **núcleo** continuará o mesmo. **Ele não "sabe" nem "se importa" com detalhes externos**, por exemplo, onde os dados estão sendo guardados ou que tecnologia está sendo usada na interface.

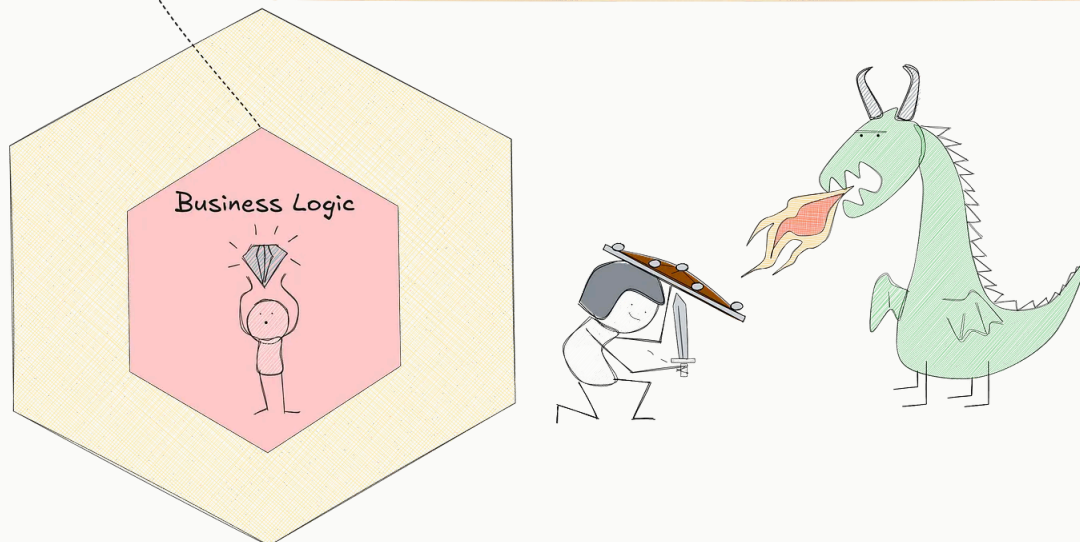
Essa separação traz vários benefícios:

1. **Fácil de Testar:** Como o núcleo é independente de tecnologias externas, ele pode ser testado de maneira isolada. Você pode garantir que as regras de negócio estão funcionando corretamente sem precisar se preocupar com banco de dados, frameworks ou APIs.
2. **Manutenção Simplificada:** Como as regras de negócio estão isoladas, fica mais fácil refatorar o código ou adicionar novas funcionalidades sem o risco de quebrar outras partes do sistema.
3. **Maior Flexibilidade:** Ao manter o núcleo livre de detalhes de implementação, você pode trocar partes do sistema sem precisar modificar as regras de negócio. Por exemplo, se você decidir mudar de um banco de dados SQL para NoSQL, o núcleo não precisa ser alterado.

Por Que o Núcleo é Importante?

Em sistemas caóticos, as regras de negócios estão espalhadas por várias camadas e misturadas com detalhes de implementação. Isso torna o sistema mais frágil e difícil de manter. Ao focar no núcleo, a Arquitetura Hexagonal resolve esse problema, permitindo que a lógica de negócios permaneça sólida e imutável, mesmo que o resto do sistema mude.

O núcleo (ou Core) da aplicação é o coração das regras de negócio, que deve ser protegido a todo custo. Aqui, o cavaleiro representa os conceitos da Arquitetura Hexagonal, que vamos explorar ao longo do artigo. Sua missão é defender o núcleo contra o dragão – uma metáfora para influências externas, acoplamentos indesejados ou antipadrões que tentam causar caos e desordem nas regras de negócio.



No próximo tópico, vamos mergulhar em um conceito fundamental da Arquitetura Hexagonal.

Driver Side e Driven Side

Depois de entender que o núcleo da aplicação contém as regras de negócio e está protegido, é hora de explorar como ele interage com o mundo externo. Para manter essa comunicação organizada e desacoplada, a **Arquitetura Hexagonal** introduz dois lados bem definidos: o **Driver Side** e o **Driven Side**. Esses conceitos nos ajudam a entender **quem inicia as ações e quem responde a elas** dentro do sistema.

O que é o Driver Side?

O **Driver Side** é responsável por **iniciar as ações** no sistema. Ele representa as **entradas** que chegam ao núcleo da aplicação, como requisições feitas por usuários, sistemas externos ou eventos que acionam a lógica de negócio. Em outras palavras, o Driver Side é quem **"dirige"** o fluxo das operações.

Aqui estão alguns exemplos do **Driver Side** em ação:

- **Interface de Usuário (UI):** Quando o usuário clica em um botão, a interface de usuário está iniciando uma ação que o núcleo da aplicação vai processar.

- **APIs Externas:** Quando um sistema externo faz uma requisição para sua API, ele está "dirigindo" a comunicação com o núcleo.
- **Eventos:** Em sistemas orientados a eventos, um evento (como uma mensagem em uma fila) pode ser o que inicia uma ação no núcleo.

Então fica claro que Driver é quem **manda as solicitações** para o núcleo e espera que ele tome as decisões com base nas regras de negócio.

O que é o Driven Side?

Se o Driver Side é o lado que **inicia as ações**, o **Driven Side** é o lado que **responde** às decisões tomadas pelo núcleo. Uma vez que o núcleo processa a solicitação, ele precisa **executar ações externas**, como salvar dados, enviar notificações ou interagir com outros sistemas. É aí que o Driven Side entra em cena.

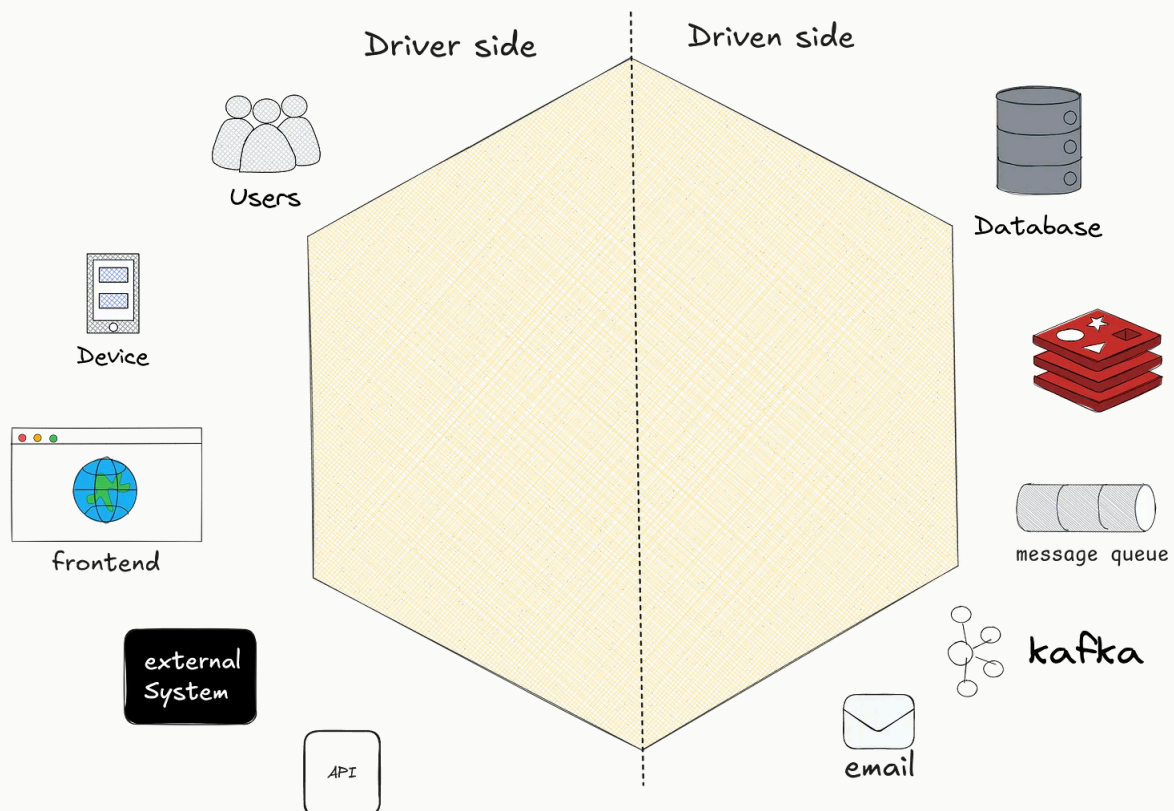
Exemplos do **Driven Side**:

- **Banco de Dados:** Quando o núcleo precisa salvar ou recuperar informações, ele faz isso através do Driven Side, sem saber exatamente como o banco de dados está implementado.
- **Serviços Externos:** Se o núcleo precisa chamar uma API de terceiros, como um serviço de pagamento, o Driven Side cuida disso.
- **Envio de E-mails:** Quando a lógica de negócio decide que um e-mail deve ser enviado, o Driven Side executa essa ação, sem expor o núcleo aos detalhes de como isso é feito.

Resumindo tudo isso, Driven Side é quem **realiza** as ações externas que o núcleo precisa, mas o núcleo permanece **independente** dos detalhes.

Por Que Essa Separação é Importante?

A separação entre **Driver Side** e **Driven Side** é crucial porque mantém o núcleo da aplicação **isolado** das implementações externas. No Driver Side, as ações vêm de fora e são encaminhadas ao núcleo. No Driven Side, o núcleo aciona as interações externas, sem precisar saber como elas acontecem.



Agora chegou o momento de falar sobre outros conceitos fundamentais dessa arquitetura!

O que são Ports na Arquitetura Hexagonal?

Na Arquitetura Hexagonal, uma Porta (ou *Port*) é como um contrato que define uma regra clara de comunicação entre o núcleo da aplicação e o mundo externo. **As Portas estabelecem "o que" precisa ser feito em cada interação, mas não especificam "como" isso será realizado.** Dessa forma, **elas garantem que o núcleo da aplicação não precise se adaptar aos detalhes das tecnologias externas,** mantendo-se focado nas regras de negócio.

Como Funciona uma Porta (Port)

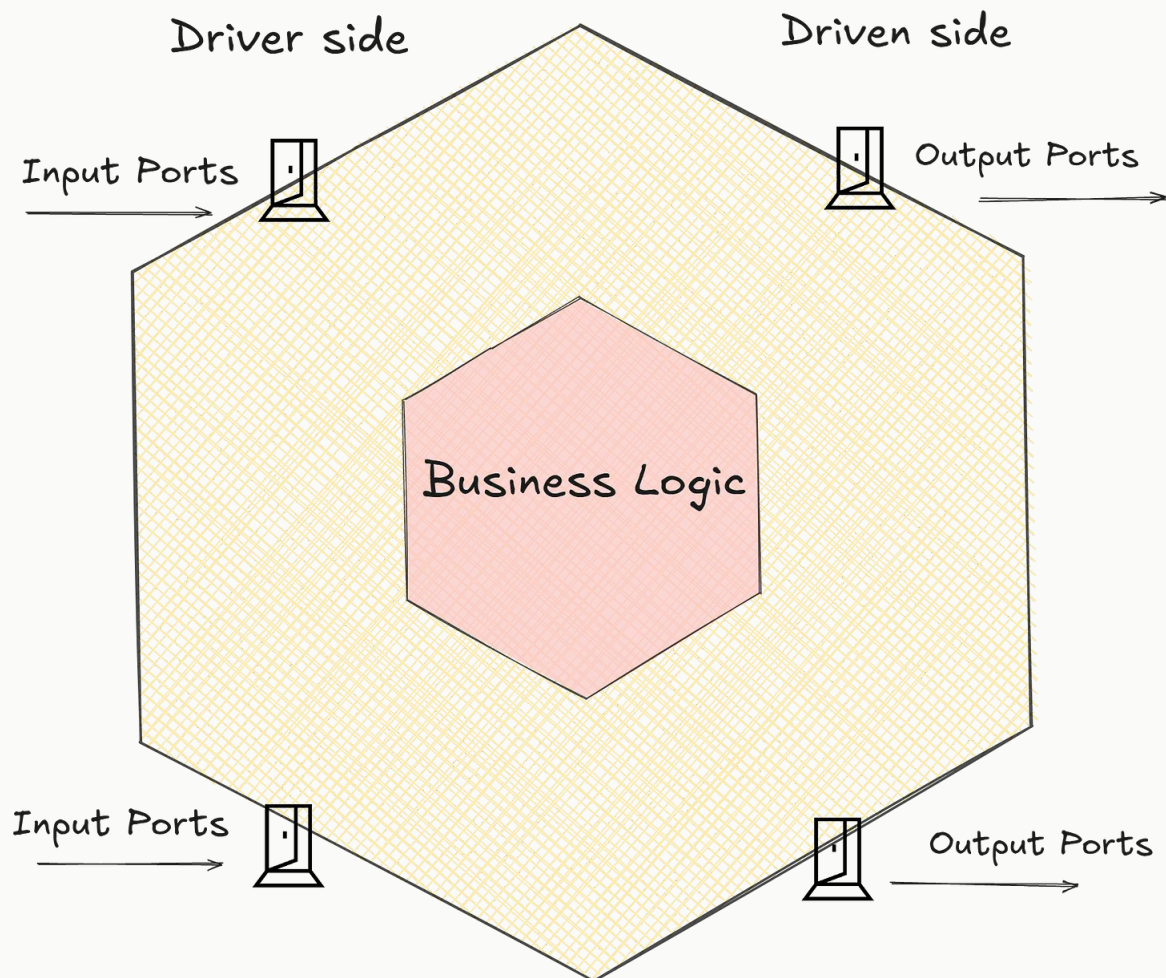
Pense nas Portas como contratos de serviço. Elas especificam que tipo de dados o núcleo espera receber e em que formato ele enviará informações, sem precisar saber de onde ou para onde esses dados vão. As Portas definem as expectativas para a comunicação, criando uma camada de proteção e independência para o núcleo da aplicação.

Porta é um Contrato!

Imagine que você contrata um serviço de entrega. Você faz um contrato que define que as entregas devem ser feitas até a porta da sua casa e que os pacotes precisam estar intactos. O contrato não se importa com como o serviço vai fazer a entrega (se de bicicleta, moto, ou carro). Esse contrato (Porta) define a "regra" da entrega, garantindo que você não precise se preocupar com os detalhes de como ela é realizada.



Na aplicação, a Porta funciona exatamente assim. Ela estabelece uma "regra de entrega" dos dados para o núcleo, permitindo que ele trabalhe de maneira independente dos detalhes de como esses dados são capturados ou enviados para o mundo externo.



Tipos de Portas na Arquitetura Hexagonal

- **Portas de Entrada (Input Ports):** São contratos que definem como as solicitações devem ser entregues ao núcleo. Se uma API externa ou um usuário envia uma solicitação, a Porta de Entrada define que ela será traduzida e entregue ao núcleo no formato esperado.
- **Portas de Saída (Output Ports):** Esses contratos definem como o núcleo envia dados para fora. Se o núcleo precisa salvar dados ou enviar uma notificação, a Porta de Saída garante que esses dados sejam enviados no formato esperado, sem precisar saber como o destino vai tratá-los.

Benefícios das Portas como Contratos na Arquitetura Hexagonal

- **Facilidade de Testes:** Como o núcleo está isolado por contratos claros, é fácil testar suas regras de negócio sem depender das tecnologias externas.
- **Adição de Funcionalidades:** Novas funcionalidades entram e saem do núcleo por meio dos contratos das Portas, mantendo o foco do núcleo nas regras de negócio.

- **Manutenção Simplificada:** Com as Portas definindo a comunicação, você pode mudar partes externas (como APIs ou bancos de dados) sem precisar alterar o núcleo.

Com as Portas funcionando como contratos, o núcleo da aplicação se mantém independente e protegido de mudanças externas, funcionando sempre dentro das expectativas definidas pelas Portas, sem se preocupar com os detalhes do mundo externo.

Vendo Portas na Prática

Agora que já discutimos o conceito de Portas na Arquitetura Hexagonal, vamos ver como elas são implementadas na prática. A ideia aqui é mostrar como o núcleo da aplicação pode ser estruturado de forma flexível, mantendo o foco em independência e desacoplamento. Abaixo, temos um exemplo que utiliza TypeScript e segue esses princípios.

Porta de Entrada (Input Port) e Porta de Saída (Output Port)

Primeiro, vamos definir nossas Portas. A Porta de Entrada define a interface para criar um novo usuário, enquanto a Porta de Saída especifica como o caso de uso salva o usuário no sistema externo.

```
// Porta de Entrada (Input Port)
export interface CreateUserPort {
  create(user: Pretify<IUserCreate>): Promise<IUserToUI>;
}

// Porta de Saída (Output Port)
export interface UserRepositoryPort {
  create(user: IUser): Promise<IUser>;
}
```

- `CreateUserPort` é a **Porta de Entrada** que inicia a ação de criação de um usuário. Ela recebe um objeto de dados de entrada (`IUserCreate`) e retorna uma representação para a interface de usuário (`IUserToUI`), mantendo o caso de uso focado apenas na lógica de criação.
- `UserRepositoryPort` é a **Porta de Saída** que permite que o caso de uso salve o usuário em um sistema externo (como um banco de dados). Ela abstrai a persistência de dados, permitindo que o `CreateUserUseCase` interaja com o repositório sem se preocupar com detalhes de implementação.

Essas Portas trabalham juntas para garantir que o núcleo da aplicação permaneça independente dos detalhes de armazenamento e de apresentação, focando-se apenas na lógica de criação do usuário.

A Fábrica de Usuários

Vamos também introduzir o conceito de uma fábrica para criar usuários. Em vez de colocar a lógica de criação e validação de um usuário diretamente no caso de uso, utilizamos uma fábrica para segregar bem as responsabilidades.

```
// Fábrica de Usuários
export class UserFactory {
  createUser(name: UserName, email: UserEmail, password: UserPassword): User {
    return new User(name, email, password);
  }
}

// Entidade User com validação de e-mail, nome e senha
export class User implements IUser {
  public id?: UserId;
  public name: UserName;
  public email: UserEmail;
  public password: UserPassword;
  public events: IUserEvent[] = [];

  constructor(name: UserName, email: UserEmail, password: UserPassword, id?: UserId) {
    if (name.length < 4) {
      throw new Error("Name must be at least 4 characters long");
    }
    if (password.length < 8) {
      throw new Error("Password must be at least 8 characters long");
    }
    if (!email.includes("@")) {
      throw new Error("Email must be valid");
    }
    this.id = id;
    this.name = name;
    this.email = email;
    this.password = password;
  }

  async domainEvents(): Promise<IUserEvent[]> {
    return this.events;
  }

  async clearEvents(): Promise<void> {
    this.events = [];
  }

  async addEvent(event: IUserEvent): Promise<void> {
    this.events.push(event);
  }
}
```

```
}  
}
```

Aqui, a `UserFactory` cria o usuário, e a entidade `User` cuida da validação, como garantir que o e-mail é válido, o nome tenha ao menos 4 caracteres e a senha 8. Isso mantém o caso de uso mais organizado e focado em sua responsabilidade.

Implementando o Caso de Uso `CreateUserUseCase`

No exemplo abaixo, o caso de uso `CreateUserUseCase` orquestra a criação e persistência de um novo usuário, utilizando Portas de Saída para manter o núcleo da aplicação independente dos detalhes externos. Esse design segue os princípios da Arquitetura Hexagonal, promovendo uma estrutura modular e flexível.

O `CreateUserUseCase` depende das seguintes Portas de Saída:

1. **UserRepositoryPort:** Define as operações necessárias para persistir o usuário em um banco de dados ou sistema externo.
2. **UserEventHandlerPort:** Lida com eventos de domínio, permitindo que o sistema responda a ações importantes, como a criação de um novo usuário.
3. **PasswordHasherPort:** Fornece uma abstração para o hashing da senha, isolando o caso de uso dos detalhes de implementação de segurança.

Essas dependências são injetadas via injeção de dependência, facilitando a substituição e simulação para testes.

```
@injectable()  
export class CreateUserUseCase implements CreateUserPort {  
  constructor(  
    @inject("UserRepository") private userRepository: UserRepositoryPort,  
    @inject("UserEventHandler") private userEventHandler: UserEventHandlerPort,  
    @inject("PasswordHasher") private passwordHasher: PasswordHasherPort  
  ) {}  
  
  async create(create: Pretify<IUserCreate>) {  
    try {  
      // Hashing da senha usando o serviço especializado de hashing  
      create.password = await this.passwordHasher.hash(create.password);  
  
      // Criação da instância de usuário com validações internas  
      const user = new User(create.name, create.email, create.password, uuidv4());  
  
      // Salvando o usuário no repositório  
      const persist = await this.userRepository.create(UserMapper.toPersistence(user));  
    }  
  }  
}
```

```

        // Emissão de evento de criação de usuário, caso o usuário seja salvo com sucesso
        if (persist.id) {
            user.addEvent(new UserCreatedEvent(persist.id));
            this.userEventHandler.handle(user.events[0]);
        }

        // Retorno de uma representação de usuário adequada para a interface de usuário
        return UserMapper.toUI(persist);
    } catch (error: any) {
        throw error; // Permite que a camada superior trate o erro conforme necessário
    }
}
}

```

Explicação do Funcionamento

Vamos ver passo a passo o que o `CreateUserUseCase` faz e como cada parte contribui para tornar o caso de uso mais coeso e independente.

1. Hashing da Senha

Em vez de lidar com o hashing da senha diretamente, o caso de uso delega essa responsabilidade ao `PasswordHasherPort`, uma interface injetada que trata exclusivamente do hashing. Com isso, o caso de uso fica livre dos detalhes técnicos de segurança, permitindo que ele se concentre em sua tarefa principal: criar o usuário.

2. Criação do Usuário

A instância de `User` é criada com as informações recebidas e com validações embutidas na classe, como a verificação de nome, e-mail e força da senha. Isso garante que o caso de uso confie totalmente na entidade `User` para gerenciar esses detalhes, sem se sobrecarregar com regras de validação.

3. Persistência no Repositório

Quando o usuário é criado com sucesso, o caso de uso utiliza a `UserRepositoryPort` para armazená-lo, delegando a responsabilidade de persistência. Com o `UserMapper`, o usuário é convertido para o formato necessário ao armazenamento, mantendo o caso de uso independente das particularidades de banco de dados.

4. Disparo de Evento de Criação

Depois que o usuário é salvo, o `CreateUserUseCase` adiciona um evento `UserCreatedEvent` ao usuário, indicando que a criação foi concluída com sucesso. Esse evento é gerenciado por um manipulador de eventos (`UserEventHandlerPort`), permitindo que o sistema reaja ao evento sem que o caso de uso precise saber como isso ocorre.

5. Retorno para a Interface de Usuário

No final, o `UserMapper` converte a entidade `User` em um formato adequado para a interface de usuário, tornando o caso de uso compatível com a camada de apresentação sem precisar conhecer seus detalhes.

Benefícios dessa Abordagem

Com essa estrutura, o `CreateUserUseCase` fica mais organizado e cada peça do sistema realiza uma tarefa específica. Veja os principais ganhos:

- **Separação de Responsabilidades:** Cada função — hashing, persistência, emissão de eventos — é tratada por um serviço especializado, o que deixa o caso de uso claro e focado na criação de usuários.
- **Facilidade para Testes:** As dependências do caso de uso são injetadas, permitindo o uso de mocks nos testes. Assim, é possível testar a lógica de criação de usuários sem precisar acessar o banco de dados ou serviços externos.
- **Escalabilidade e Flexibilidade:** Como tudo está modularizado, novas funcionalidades, como adicionar eventos ou novos serviços, podem ser integradas sem que o caso de uso precise ser alterado. Isso torna o sistema mais flexível e fácil de expandir.

Essa implementação do `CreateUserUseCase` exemplifica como a Arquitetura Hexagonal promove um design limpo e orientado a responsabilidades. O resultado é um sistema que se mantém organizado, sustentável e pronto para evoluir conforme as necessidades crescem.

E os Adaptadores? Vamos Detalhar Melhor!

Agora que entendemos o papel das Portas como contratos, podemos avançar para os *Adapters*. Se as Portas definem as "regras de entrega" entre o núcleo e o mundo externo, os Adapters são os executores dessas regras. Eles entram em cena para garantir que o contrato da Porta seja cumprido, organizando e formatando os dados adequadamente para que o núcleo possa processá-los ou enviar uma resposta.

Quem São os Adapters?

Pense nos Adapters como o serviço de entrega no exemplo da encomenda. O contrato (a Porta) define que a entrega deve ser feita até sua porta e em perfeitas condições, mas quem realmente executa essa entrega é o serviço contratado. Ele

cuida dos detalhes práticos – seja por carro, moto ou bicicleta – e garante que tudo chegue conforme as regras definidas pelo contrato.



No sistema, os Adapters atuam exatamente assim: eles pegam os dados vindos do mundo externo, organizam, formatam e traduzem essas informações no formato necessário para o núcleo da aplicação. Eles também fazem o caminho inverso, preparando os dados do núcleo para serem enviados de volta ao exterior.

Como Funcionam os Adapters no Sistema?

Quando o núcleo da aplicação precisa receber dados, os Adapters entram em ação para transformar as informações do formato externo para o formato interno que o núcleo entende. Por exemplo:

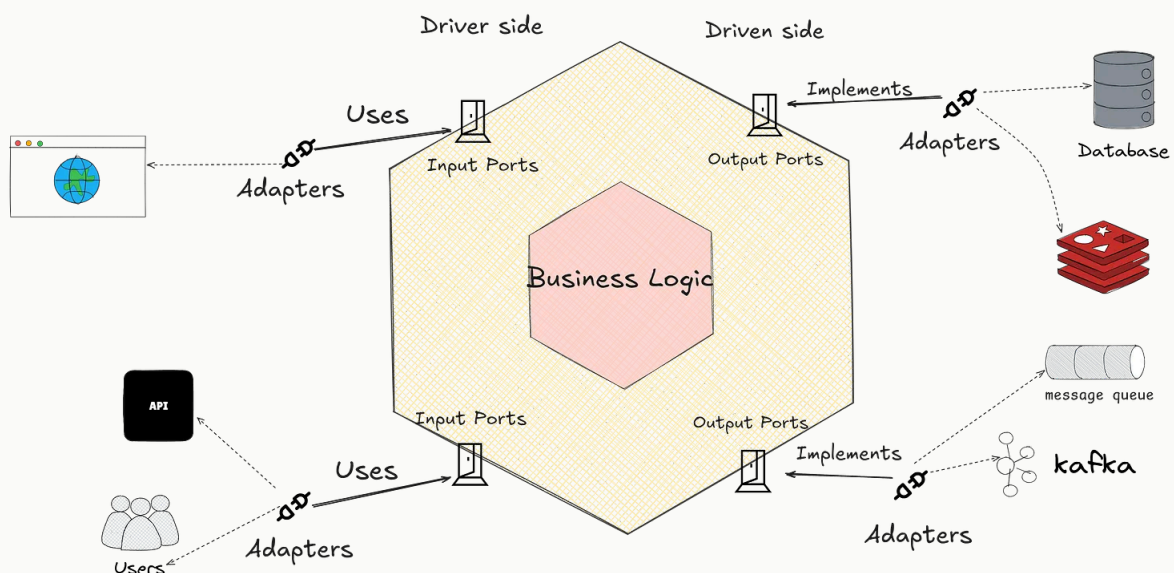
- **Recebendo uma Mensagem JSON:** Imagine que uma API externa envia uma mensagem em formato JSON. O Adapter deserializa (converte) essa mensagem, ajustando-a ao formato que o núcleo da aplicação entende. O núcleo então processa esses dados, sem saber que vieram em JSON – ele só vê as informações organizadas e prontas para uso.
- **Enviando uma Resposta:** Quando o núcleo processa uma solicitação e precisa enviar uma resposta para fora, o Adapter pega essa resposta, transforma-a no

formato adequado (como uma resposta HTTP ou uma mensagem para um sistema de mensageria), e envia ao destino.

Onde os Adapters Aparecem no Código?

No contexto do nosso exemplo de código, imagine que temos um caso de uso como o `CreateUserUseCase`, que define a lógica de criação de um novo usuário. Esse caso de uso utiliza Portas para comunicar suas necessidades ao sistema externo, mas os Adapters são as implementações reais dessas Portas:

- **Adapter para o Banco de Dados** (Porta de Saída): Esse Adapter conecta o núcleo ao banco de dados, permitindo salvar usuários de forma concreta, mas isolando o núcleo dos detalhes específicos do banco utilizado.
- **Adapter para a API REST** (Porta de Entrada): Esse Adapter traduz as requisições vindas de uma interface de usuário ou de uma API externa, organizando os dados para que o núcleo possa processá-los.



Os Adapters são os responsáveis pela "entrega e coleta" dos dados, conforme definido pelos contratos das Portas. Eles cuidam de transformar e formatar as informações para que o núcleo da aplicação funcione sem interrupções, mantendo-o isolado dos detalhes externos e permitindo que todo o sistema evolua de forma organizada.

Implementando os Adapters para as Portas

Agora que já definimos as Portas de Entrada e Saída, vamos explorar como os Adapters as implementam, conectando o núcleo da aplicação ao mundo externo, sem que o núcleo precise se preocupar com detalhes específicos de comunicação ou armazenamento.

Adapter de Banco de Dados

Para salvar um usuário em um banco de dados real, criamos um Adapter que implementa a **Porta de Saída** `UserRepositoryPort`. Esse Adapter sabe como interagir com o banco de dados, mas o núcleo da aplicação não precisa conhecer esses detalhes, o que mantém o design limpo e modular.

```
import 'reflect-metadata';
import { inject, injectable } from 'tsyringe';
import { UserRepositoryPort } from
'../../application/User/port/secondary/UserRepositoryPort';
import { IUser } from '../../application/User/domain/IUser';
import { User } from '../../application/User/domain/User';
import db from '../../infrastructure/db/db';
import { PrismaClient } from '@prisma/client';
import { UncaughtError } from "../../Errors/Uncaught";

@injectable()
export class UserRepository implements UserRepositoryPort {
  private db: PrismaClient;
  private model: typeof db.user;

  constructor() {
    this.db = db;
    this.model = this.db.user;
  }

  async create(user: IUser) {
    try {
      const exists = await this.model.findUnique({ where: { email: user.email } });
      if (exists) {
        throw new UncaughtError('user already exists', 400);
      }
      const newUser = await this.model.create({
        data: {
          name: user.name,
          email: user.email,
          password: user.password,
          id: user.id
        }
      });
      return new User(newUser.name, newUser.email, newUser.password, newUser.id);
    } catch (error: any) {
      throw new UncaughtError(error.message);
    }
  }

  async findById(id: string) {
    try {
      const user = await this.model.findUnique({ where: { id: id } });
      return user ? new User(user.name, user.email, user.password, user.id) : null;
    } catch (error: any) {
```

```

        throw new UnCaughtError(error.message);
    }
}

async findByEmail(email: string) {
    try {
        const user = await this.model.findUnique({ where: { email: email } });
        return user ? new User(user.name, user.email, user.password, user.id) : null;
    } catch (error: any) {
        throw new UnCaughtError(error.message);
    }
}
}
}

```

Nesse exemplo, `UserRepository` **implementa a Porta de Saída** `UserRepositoryPort`. Ele interage com o banco de dados usando Prisma para operações de persistência e recuperação de dados, mas o núcleo da aplicação não precisa saber desses detalhes, apenas delega ao Adapter.

Vantagens de Utilizar Adapters com Interfaces Especializadas

Estruturar o design com interfaces especializadas traz vários benefícios além de organização:

1. Responsabilidades Bem Definidas:

- Ao delegar cada ação a uma interface específica (como `UserRepositoryPort` para persistência), o código se mantém coeso e organizado. Cada interface lida com um propósito específico, facilitando a leitura e a manutenção.

2. Facilidade para Fazer Mudanças:

- Se for necessário trocar o banco de dados ou adicionar novas funcionalidades, é possível alterar apenas a implementação do Adapter sem modificar o núcleo. Isso traz flexibilidade e evita grandes refatorações.

3. Testes Simplificados:

- Separar responsabilidades facilita o uso de mocks e testes independentes, eliminando a necessidade de configurar um banco de dados real em cenários que não envolvem persistência direta.

4. Adaptabilidade:

- Com um design modular, é fácil adicionar funcionalidades novas, como salvar dados em múltiplos bancos de dados ou enviar informações para

uma fila de mensagens, apenas criando novas implementações da interface de saída.

Adapter de API REST

Agora, imagine que você esteja recebendo uma requisição de uma API REST para criar um usuário. Neste caso, você precisará de um Adapter que **use a Porta de Entrada** `CreateUserPort`, manipulando a comunicação externa, como a deserialização dos dados recebidos da API e a passagem desses dados para o núcleo da aplicação.

```
import 'reflect-metadata';
import { inject, injectable } from 'tsyringe';
import { CreateUserPort } from '../../application/User/port/primary/CreateUserPort';
import { IUser } from '../../application/User/domain/IUser';
import { UserMapper } from '../../mappers/UserMapper';

@injectable()
export class UserCreateController {
  private userMapper: typeof UserMapper;

  constructor(@inject('CreateUserUseCase') private userCreate: CreateUserPort) {
    this.userMapper = UserMapper;
  }

  async create(body: { name: string; password: string; email: string }): Promise<any> {
    try {
      const userDTO = this.userMapper.toDomain(body);
      const user = await this.userCreate.create({
        name: userDTO.name,
        password: userDTO.password,
        email: userDTO.email
      });
      return user;
    } catch (error) {
      throw error;
    }
  }
}
```

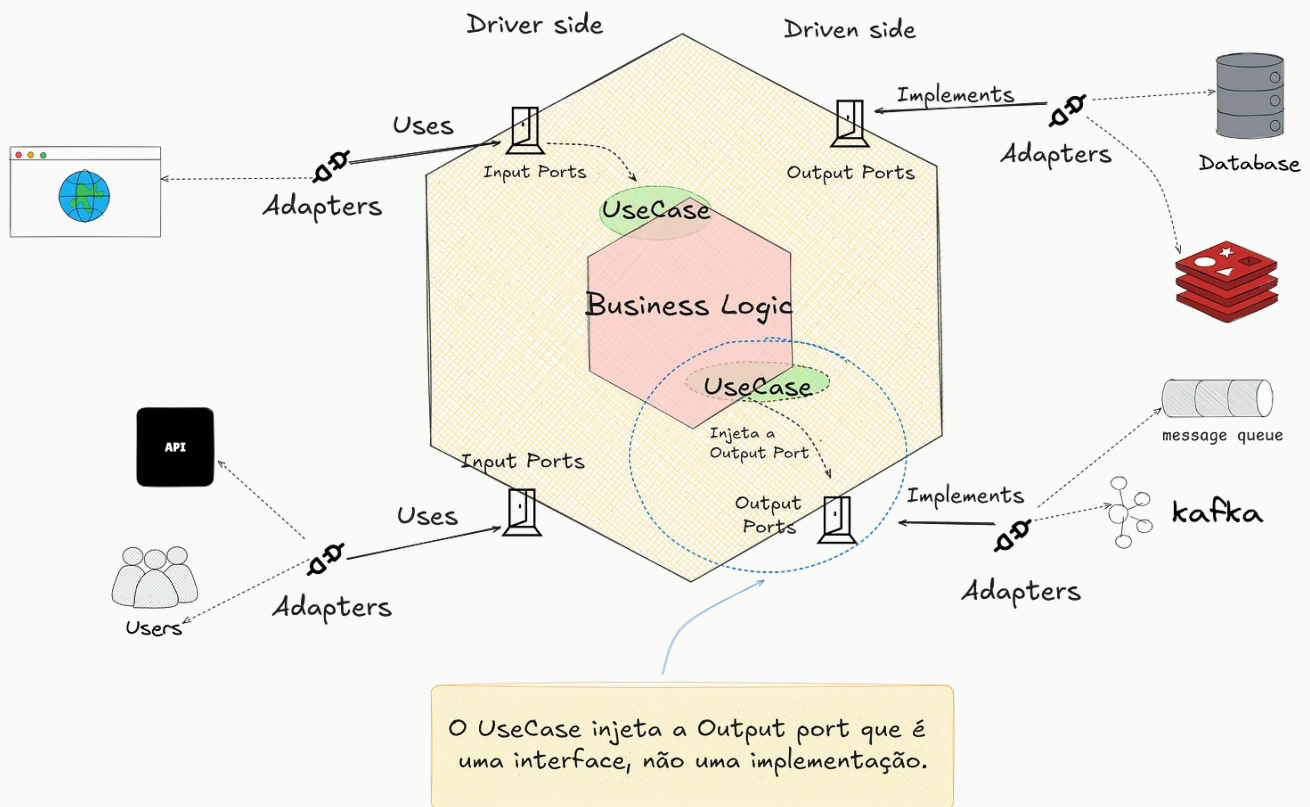
No código acima, `UserCreateController` é o Adapter que usa a **Porta de Entrada** `CreateUserPort`. Ele recebe as solicitações de criação de usuário da API, converte os dados para o formato adequado com o `UserMapper`, e então chama o caso de uso (`CreateUserUseCase`). Com isso, o Adapter cuida da comunicação externa e o núcleo permanece focado apenas nas regras de negócio.

Essa estrutura modular torna o sistema adaptável e escalável, pronto para responder a mudanças e a novas exigências sem comprometer a coesão ou o

design da aplicação.

Como os Adapters Ajudam na Flexibilidade e Agilidade das Entregas

A imagem destaca o papel dos *Adapters* como implementações das Portas, conectando o núcleo da aplicação (onde estão os *use cases* e a lógica de negócios) ao mundo externo.



Vamos ver alguns pontos-chave que mostram como essa estrutura proporciona flexibilidade e agilidade:

1. **Separação de Responsabilidades:** Os Adapters conectam o núcleo da aplicação a sistemas externos (banco de dados, APIs, etc.), mantendo o núcleo focado apenas nas regras de negócio, sem depender de detalhes técnicos externos.
2. **Portas como Contratos:** As Portas atuam como contratos entre o núcleo e o mundo externo. As Portas de Entrada permitem que o núcleo receba informações, e as Portas de Saída permitem que ele envie dados. Essa estrutura deixa as implementações concretas sob a responsabilidade dos Adapters.
3. **Injeção de Dependências para Flexibilidade:** Use cases dependem de interfaces (Portas) injetadas, o que permite trocar os Adapters sem modificar

o núcleo. Isso facilita a substituição de sistemas externos, como bancos de dados, de forma rápida.

4. **Facilidade de Testes:** Mocks ou stubs das Portas podem ser usados para testar a lógica de negócios do núcleo isoladamente, sem dependências externas, acelerando o desenvolvimento e a entrega contínua.
5. **Adição de Funcionalidades:** Novas funcionalidades podem ser implementadas criando novos Adapters que seguem o contrato das Portas, sem alterar o núcleo da aplicação. Isso facilita a expansão do sistema de forma rápida e segura.

A grande vantagem aqui é que, com Adapters, sua aplicação está sempre pronta para crescer e se adaptar. Sem dramas e sem complexidade. Agora vejamos os testes para o UseCase.

Testando o Caso de Uso

Para garantir que o `CreateUserUseCase` está funcionando corretamente, vamos criar um teste utilizando mocks para simular o comportamento das dependências injetadas. Com isso, focamos no núcleo da aplicação, testando a lógica de negócio de maneira independente dos Adapters externos.

Vamos configurar os mocks para simular o comportamento do `UserRepositoryPort` e do `PasswordHasherPort`.

```
describe("CreateUserUseCase", () => {
  let mockUserRepository: jest.Mocked<UserRepositoryPort>;
  let mockUserEventHandler: jest.Mocked<UserEventHandlerPort>;
  let mockPasswordHasher: jest.Mocked<PasswordHasherPort>;
  let createUserUseCase: CreateUserUseCase;

  beforeEach(() => {
    // Configurando mocks para dependências
    mockUserRepository = {
      create: jest.fn(),
      findById: jest.fn(),
      findByEmail: jest.fn(),
    };
    mockUserEventHandler = { handle: jest.fn() };
    mockPasswordHasher = { hash: jest.fn().mockResolvedValue("hashed_password") };

    createUserUseCase = new CreateUserUseCase(mockUserRepository, mockUserEventHandler,
    mockPasswordHasher);
  });

  it("deve criar e salvar o usuário no repositório", async () => {
    const userData: Pretify<IUserCreate> = { name: "John Doe", email: "john.doe@example.com",
    password: "password123" };
  });
});
```

```

const persistedUser = { ...userData, password: "hashed_password", id: "generated_id" };

// Arrange - Mock do comportamento
mockUserRepository.create.mockResolvedValue(persistedUser);

// Act
const result = await createUserUseCase.create(userData);

// Assert
expect(mockPasswordHasher.hash).toHaveBeenCalledWith("password123");
expect(mockUserRepository.create).toHaveBeenCalledWith(expect.objectContaining({ name: "John Doe", email: "john.doe@example.com" }));
expect(result).toEqual(expect.objectContaining({ id: "generated_id", name: "John Doe", email: "john.doe@example.com" }));
});

it("deve emitir o evento 'user_created' ao criar o usuário", async () => {
  const userData: Pretify<IUserCreate> = { name: "John Doe", email: "john.doe@example.com", password: "password123" };
  const persistedUser = { ...userData, password: "hashed_password", id: "generated_id" };

  // Arrange
  mockUserRepository.create.mockResolvedValue(persistedUser);

  // Act
  await createUserUseCase.create(userData);

  // Assert
  expect(mockUserEventHandler.handle).toHaveBeenCalledWith(expect.objectContaining({ type: "user_created", payload: { id: "generated_id" } }));
});

```

Pontos Fortes do Teste

1. Isolamento da Lógica de Negócio:

- Com a Arquitetura Hexagonal, as dependências externas, como o repositório, o serviço de eventos e o hasher de senhas, são injetadas no caso de uso (`CreateUserUseCase`). Isso permite que, durante os testes, cada dependência seja substituída por um mock.
- Esse isolamento significa que estamos testando apenas a lógica de criação do usuário, sem interagir com sistemas reais, como banco de dados ou serviços de autenticação.

2. Mocks das Dependências:

- Ao utilizar mocks para `UserRepositoryPort`, `UserEventHandlerPort`, e `PasswordHasherPort`, o teste consegue focar nos métodos e fluxos que

realmente interessam, assegurando que a lógica de negócios não dependa de implementações externas.

- Com mocks, o teste é rápido e eficiente, pois não precisa de configurações complexas de ambiente, e facilita o teste de cada caso específico, como a criação do usuário, o envio de evento e o lançamento de erros.

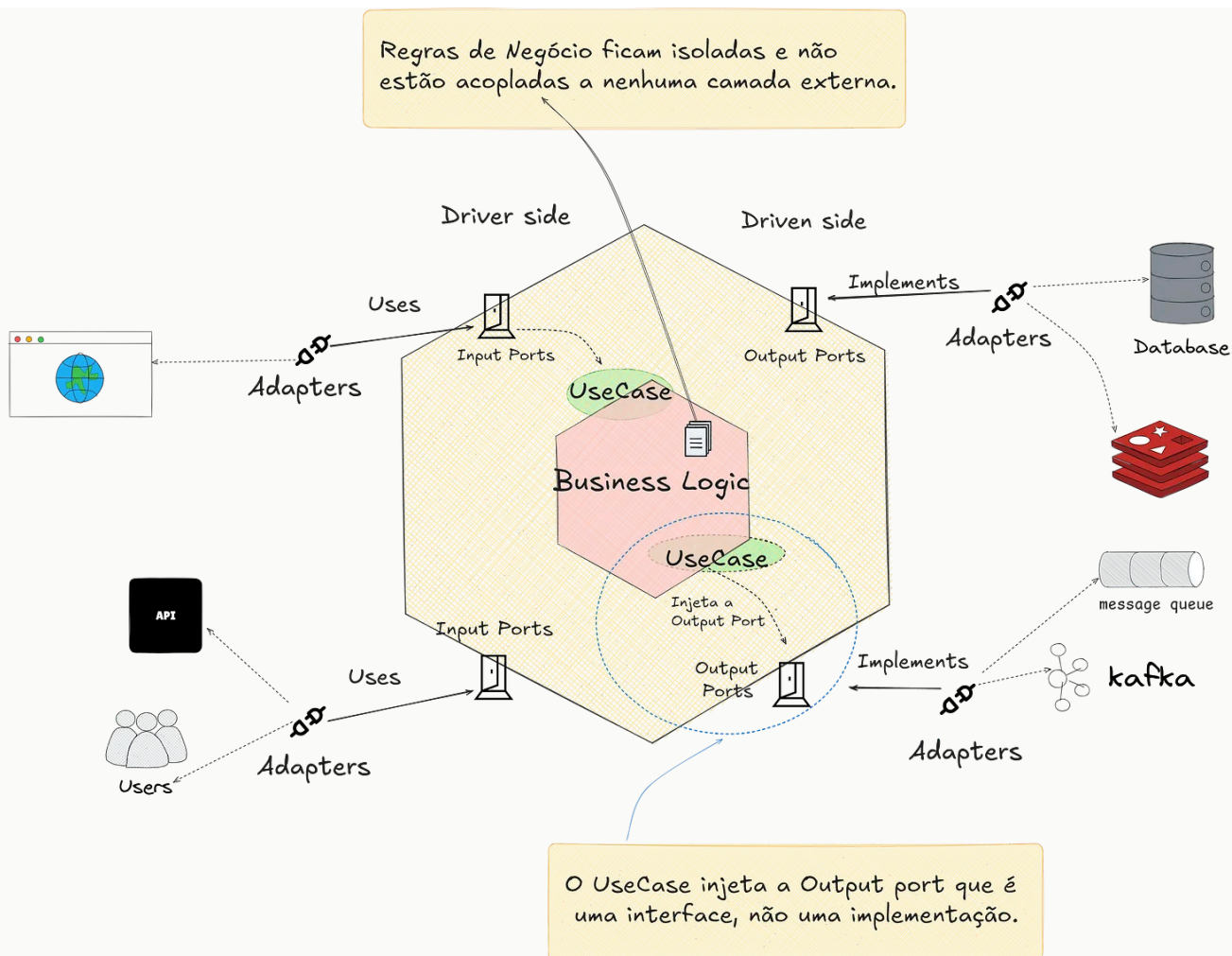
3. Verificação de Comportamento das Dependências:

- Cada assert verifica uma chamada específica, como o uso de hashing, a persistência do usuário e a emissão do evento de criação. Isso garante que o `CreateUserUseCase` está interagindo corretamente com cada dependência.
- A arquitetura hexagonal, ao definir portas para cada dependência, permite que o teste valide esses comportamentos com precisão, o que melhora a qualidade e a confiança no código.

Agora vamos conversar brevemente sobre algumas questões importantes!

Diferença entre Injeção e Implementação nas Portas de Entrada e Saída

Na Arquitetura Hexagonal, é essencial entender como as **Portas de Entrada e Saída** operam e como **Use Cases** e **Adapters** interagem com elas. A forma como **injeção** e **implementação** ocorrem varia entre o **Driver Side** e o **Driven Side**. Veja a imagem novamente.



Vamos explorar essa distinção de forma prática com o diagrama.

Driver Side (Portas de Entrada)

- **Adapters Usam as Portas de Entrada:** No Driver Side (à esquerda no diagrama), os Adapters recebem solicitações externas (como de uma interface de usuário ou API) e **usam as Portas de Entrada** para ativar a lógica de negócio. Eles acionam o núcleo da aplicação através das Portas de Entrada.
 - **Exemplo:** `UserCreateController` usa a Porta `CreateUserPort` para acionar o `CreateUserUseCase`, onde ocorre a lógica de criação de um usuário.
- **Use Cases Implementam as Portas de Entrada:** Os Use Cases são responsáveis pela lógica de negócio e **implementam as Portas de Entrada**. Isso define o contrato que o Adapter segue para chamar as funcionalidades necessárias.
 - **Exemplo:** `CreateUserUseCase` implementa a interface `CreateUserPort`, permitindo que o Adapter o utilize para criar um usuário.
- **Injeção no Driver Side:** Os Use Cases não injetam Portas de Entrada, pois eles **mesmos as implementam**. Em vez disso, os Adapters injetam os Use Cases.

para executar as ações, sem precisar conhecer os detalhes internos da lógica.

Driven Side (Portas de Saída)

- **Adapters Implementam as Portas de Saída:** No Driven Side (à direita no diagrama), os Adapters fornecem as funcionalidades externas que o núcleo precisa, **implementando as Portas de Saída**. Eles conectam o núcleo a recursos externos, como bancos de dados ou sistemas de mensagens.
 - **Exemplo:** `UserRepositoryAdapter` implementa `UserRepositoryPort`, permitindo que o Use Case interaja com o banco de dados para salvar o usuário.
- **Use Cases Injetam as Portas de Saída:** No Driven Side, os Use Cases injetam Portas de Saída para realizar operações externas de forma desacoplada, sem conhecer detalhes da implementação. Isso permite que o núcleo funcione sem depender de serviços externos específicos.
 - **Exemplo:** `CreateUserUseCase` injeta `UserRepositoryPort` para salvar um usuário, mas não precisa saber onde ou como isso ocorre.

Esse modelo de injeção e implementação nas Portas de Entrada e Saída permite que o sistema seja escalável, modular e fácil de manter. O núcleo da aplicação foca apenas na lógica de negócio, enquanto os Adapters lidam com as interações externas. Essa separação de responsabilidades é essencial para um sistema flexível e pronto para mudanças, mantendo um design coeso ao longo do tempo.

Existe uma regra rígida sobre o número de camadas na Arquitetura Hexagonal?

Bom, a resposta simples é: **não**, não existe uma regra rígida sobre quantas camadas você deve ter na Arquitetura Hexagonal. O objetivo central dessa arquitetura é **desacoplar o núcleo da aplicação** do mundo externo usando **Portas** e **Adaptadores**, mas o número exato de camadas pode variar dependendo do projeto.

E o que é comum em livros e na prática?

Se olharmos para os livros, artigos e palestras sobre **Arquitetura Limpa** e **Arquitetura Hexagonal** – como o de **Alistair Cockburn** ou o famoso **Robert C. Martin (Uncle Bob)** com sua **Clean Architecture** – não existe uma receita mágica sobre o número de camadas. A grande questão aqui é sempre a **flexibilidade** e o **isolamento** do núcleo, não o número exato de camadas.

Na prática, em muitos projetos, o que a gente vê é uma organização com algo em torno de **4 a 5 camadas principais**. Isso, claro, depende do nível de complexidade e abstração que o projeto precisa. Geralmente, essas camadas incluem:

- **Camada de Núcleo (Core):** Onde ficam as **regras de negócio** e as **entidades**.
- **Portas (Ports):** As **interfaces** que expõem as funcionalidades internas e definem como o núcleo se comunica com o mundo externo.
- **Adaptadores (Adapters):** São as implementações das Portas, que conectam o núcleo com a infraestrutura.
- **Infraestrutura (Infrastructure):** Onde estão as **configurações** de banco de dados, APIs, filas de mensagens, entre outros.

Em alguns projetos, pode haver uma camada adicional de **Serviços Externos** ou algo como uma **Integration Layer**, onde você lida diretamente com serviços de terceiros, APIs externas, ou integrações complexas. Mas isso é mais comum em sistemas maiores ou com muitas dependências externas.

No fim das contas, **não existe um número fixo de camadas** na Arquitetura Hexagonal. O mais importante é a **flexibilidade** para adaptar o sistema às necessidades do projeto.

Fico por aqui neste post! Se gostar do conteúdo compartilhe! Até o próximo! 😊



21 Likes · 1 Restack

Discussão sobre este post

Comentários Restacks



Escreva um comentário...



Darlison Limeira 3 de nov. de 2024

...

♥ Curtido por Rafael Miguel

Ótimo artigo! Acho que esse foi o melhor li até agora sobre o tema.

Tem o repo do exemplo no github?

♥ LIKE (1) 💬 RESPONDER

🔗 COMPARTILHAR

2 respostas de Rafael Miguel e outros



Joao Victor 19 de dez. de 2024



Artigo ficou muito bom, bem escrito, de fácil leitura e bem rica em conteúdo com exemplos e imagens ilustrativas, o que ajuda no processo de entendimento da arquitetura.

♡ LIKE (1) 💬 RESPONDER

🔗 COMPARTILHAR

1 resposta de Rafael Miguel

3 comentários a mais...